Servlets

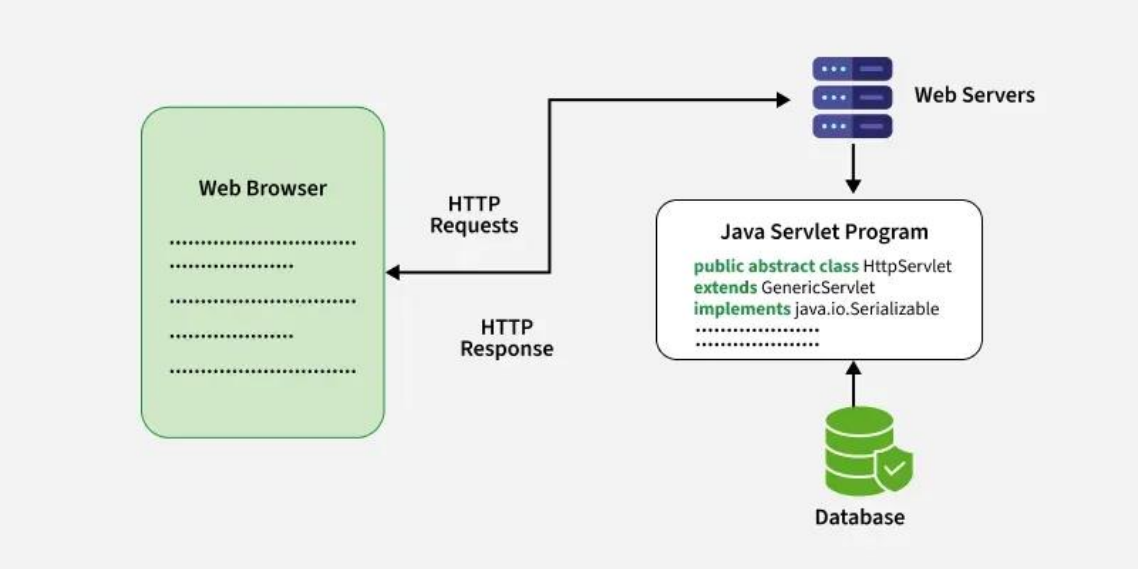
Java Servlet is a Java program that runs on a Java-enabled web server or application server. It handles client requests, processes them and generates responses dynamically. Servlets are the backbone of many server-side Java applications due to their efficiency and scalability.

Features of Java Servlets

- Work on the server-side.
- Efficiently handle complex client requests.
- Generate dynamic responses.
- Provide better performance compared to older technologies like CGI.
- Highly scalable in enterprise-level web applications.

Java Servlets Architecture: Java servlets container plays a very important role. It is responsible for handling important tasks like load balancing, session management and resource allocation, it makes sure that all the requests are process efficiently under high traffic. The container distributes requests across multiple instances, which helps improve the system performance.

Servlet Architecture can be depicted from the image itself as provided below as follows:



Servlet Architecture Workflow:

Execution of Servlets basically involves Six basic steps:

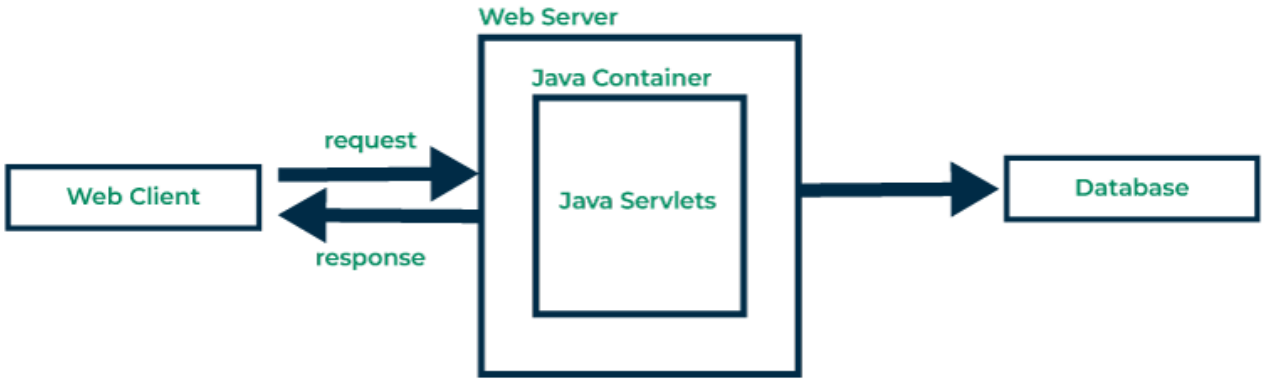
- The Clients send the request to the Web Server.
- The Web Server receives the request.
- The Web Server passes the request to the corresponding servlet.
- The Servlet processes the request and generates the response in the form of output.
- The Servlet sends the response back to the webserver.
- The Web Server sends the response back to the client and the client browser displays it on the screen.

Need of Server-Side Extensions

- Server-side extensions allow dynamic web page generation by running programs on the server.
- Web servers provide APIs to help developers build these applications.
- Java Servlets (part of Jakarta EE) are a key API for Java-based web development.

Servlet Container

Servlet container, also known as Servlet engine, is an integrated set of objects that provide a run time environment for Java Servlet components. It is a system that manages Java Servlet components on top of the Web server to handle the Web client requests.



Services provided by the Servlet container:

- **Network Services:** Loads a Servlet class. The loading may be from a local file system, a remote file system or other network services. The Servlet container provides the network services over which the request and response are sent.
- **Decode and Encode MIME-based messages**: Provides the service of decoding and encoding MIME-based messages.
- **Manage Servlet container:** Manages the lifecycle of a Servlet.
- **Resource management**: Manages the static and dynamic resources, such as HTML files, Servlets and JSP pages.
- **Security Service:** Handles authorization and authentication of resource access.
- **Session Management:** Maintains a session by appending a session ID to the URL path

In Java, there are two primary types of servlets:

- Generic Servlets:
    - These servlets extend the javax.servlet.GenericServlet abstract class.
    - They are protocol-independent, meaning they can be used to handle requests from various protocols, not just HTTP.
    - The core method for handling requests in a Generic Servlet is the service() method, which must be overridden to provide specific request handling logic.
    - Generic Servlets are suitable for situations where the communication protocol might not be HTTP, or when a more generalized approach to request handling is desired.
- HTTP Servlets:
    - These servlets extend the javax.servlet.http.HttpServlet abstract class.
    - They are specifically designed to handle HTTP requests and responses, providing built-in support for the HTTP protocol.
    - HTTP Servlets provide specialized methods for handling different HTTP request types (e.g., GET, POST, PUT, DELETE), such as doGet(), doPost(), doPut(), and doDelete(). The service() method in HttpServlet automatically dispatches requests to the appropriate doXxx() method based on the HTTP method.
    - HTTP Servlets are the most commonly used type of servlet for developing web applications, as they directly address the needs of web-based communication.

In summary, the choice between Generic Servlets and HTTP Servlets depends on whether the application requires protocol independence or specific HTTP protocol handling. HTTP Servlets are generally preferred for web development due to their specialized HTTP support.

Creating a Basic Servlet

Prerequisites

- *Install JDK (Java Development Kit)*
- *Install Apache Tomcat server (e.g., version 9 or 10)*
- *Install an IDE like Eclipse or NetBeans or IntelliJ IDEA*
- *Download the Servlet API JAR (already included in Tomcat)*

Example: Here's a simple example of how a servlet works:

Step 1: Create a Dynamic Web Project (in Eclipse)

- Open Eclipse -> File ->New ->Dynamic Web Project
- Name the project (e.g., HelloWorldServlet)
- Target runtime -> Select Apache Tomcat
- Click Finish

Step 2: Create Servlet class

- Right-click on src -> New-> Servlet
- Name it HelloWorldServlet and click Finish

HelloWorldServlet.java

```
import java.io.*;

import jakarta.servlet.*;

import jakarta.servlet.http.*;


public class HelloWorldServlet extends HttpServlet {

    protected void doGet(HttpServletRequest request, HttpServletResponse
response)
```

```
    throws ServletException, IOException {
        response.setContentType("text/html");
        PrintWriter out = response.getWriter();
        out.println("<html><body><h1>Hello, World!</h1></body></html>");
    }
}
```

Explanation:

- HelloWorldServlet extends HttpServlet.
- doGet() handles HTTP GET requests.
- response.setContentType("text/html") specifies response type.
- PrintWriter sends HTML response back to the client.

Configuring a Servlet

To deploy a servlet, you need to configure it in the web.xml file. This file maps URLs to servlets. For example,

XML-Based Configuration (web.xml):

<web-app
xmlns="http://www.oracle.com/webfolder/technetwork/jsc/xml/ns/javaee/index.html"

xmlns:xsi="https://www.w3.org/2001/XMLSchema-instance"

xsi:schemaLocation="http://www.oracle.com/webfolder/technetwork/jsc/xml/ns/javaee/index.html

http://www.oracle.com/webfolder/technetwork/jsc/xml/ns/javaee/index.html/web-app_3_0.xsd"

version="3.0">

<servlet>

<servlet-name>HelloWorldServlet</servlet-name>

<servlet-class>HelloWorldServlet</servlet-class>

</servlet>

<servlet-mapping>

<servlet-name>HelloWorldServlet</servlet-name>

<url-pattern>/hello</url-pattern>

</servlet-mapping>

</web-app>

Explanation:
This is a web.xml file used for mapping URLs to servlets. So, when you visit http://localhost:8080/yourApp/hello, the servlet runs and shows "Hello, World!" in the browser.

Annotation-Based Configuration (Modern Approach)

From Servlet 3.0, servlet configuration can also be done using annotations. Instead of using web.xml, we can configure the servlet using the @WebServlet annotations.

@WebServlet("/hello")

public class HelloWorldServlet extends HttpServlet {

protected void doGet(HttpServletRequest request, HttpServletResponse response) **throws** ServletException, IOException {

```
        response.setContentType("text/html");
        PrintWriter out = response.getWriter();
        out.println("<html><body><h1>Hello, World!</h1></body></html>");
    }
}
```

Explanation: Here we have used the @WebServlet("/hello") annotation to register the servlet directly in code (no need for web.xml). It maps the servlet to the URL /hello.

Why Choose Java Servlet over other Technologies?

Dynamic web content requires server-side technologies. While there are many options, Java Servlet stand out due to their advantages over alternatives like Common Gateway Interface (CGI)

Limitations of CGI:

- Process Overhead: CGI creates and destroys a process for every client request, leading to high resource consumption.
- Scalability Issues: Poor performance with increased client requests.

Difference Between Java Servlets and CGI

The table below demonstrates the difference between servlet and CGI

Servlet	CGI (Common Gateway Interface)
Servlets are portable and efficient.	CGI is not portable.
In Servlets, sharing data is possible.	In CGI, sharing data is not possible.

Servlet	CGI (Common Gateway Interface)
Servlets can directly communicate with the webserver.	CGI cannot directly communicate with the webserver.
Servlets are less expensive than CGI.	CGI is more expensive than Servlets.
Servlets can handle the cookies.	CGI cannot handle the cookies.

Key Classes and Interfaces

Various classes and interfaces present in these packages are:

Component	Type	Package
Servlet	Interface	jakarta.servlet.*
ServletRequest	Interface	jakarta.servlet.*
ServletResponse	Interface	jakarta.servlet.*
GenericServlet	Class	jakarta.servlet.*
HttpServlet	Class	jakarta.servlet.http.*
HttpServletRequest	Interface	jakarta.servlet.http.*

Component	Type	Package
HttpServletResponse	Interface	jakarta.servlet.http.*
Filter	Interface	jakarta.servlet.*
ServletConfig	Interface	jakarta.servlet.*

Difference between HTTP GET and HTTP POST

GET	POST
In GET method we can not send large amount of data rather limited data of some number of characters is sent because the request parameter is appended into the URL.	In POST method large amount of data can be sent because the request parameter is appended into the body.
GET request is comparatively better than Post so it is used more than the Post request.	POST request is comparatively less better than Get method, so it is used less than the Get request.
GET requests are only used to request data (not modify)	POST requests can be used to create and modify data.
GET request is comparatively less secure because the data is exposed in the URL bar.	POST request is comparatively more secure because the data is not exposed in the URL bar.

GET	POST
Request made through GET method are stored in Browser history.	Request made through POST method is not stored in Browser history.
GET method request can be saved as bookmark in browser.	POST method request can not be saved as bookmark in browser.
Request made through GET method are stored in cache memory of Browser.	Request made through POST method are not stored in cache memory of Browser.
Data passed through GET method can be easily stolen by attackers as the data is visible to everyone.GET requests should never be used when dealing with sensitive data	Data passed through POST method can not be easily stolen by attackers as the URL Data is not displayed in the URL
In GET method only ASCII characters are allowed.	In POST method all types of data is allowed.
In GET method, the Encoding type is *application/x-www-form-urlencoded*	In POSTmethod, the encoding type is *application/x-www-form-urlencoded* or *multipart/form-data*. Use multipart encoding for binary data

Life Cycle of a Servlet

A Servlet is a Java program that runs on a server to handle client requests and generate dynamic responses. The Servlet life cycle is entirely managed by the Servlet container, which uses the jakarta.servlet.Servlet interface to manage Servlet objects. Understanding the life cycle helps in writing efficient and resource-safe web applications.

Note: *In Jakarta EE 9 and above, use import jakarta.servlet.* instead of javax.servlet.**

States of the Servlet Life Cycle

The Servlet life cycle consists of four main stages:

- Loading a Servlet
- Initializing a Servlet
- Handling Client Requests
- Destroying a Servlet

1. Loading a Servlet

The first stage of the Servlet lifecycle involves loading and initializing the Servlet. The Servlet container performs the following operations:

- **Loading:** The Servlet container loads the Servlet class into memory.
- **Instantiation:** The container creates an instance of the Servlet using the no-argument constructor.

When is a Servlet loaded?

- During application startup (if configured in deployment descriptor).
- On first client request (lazy loading).

2. Initializing a Servlet

After instantiation, the container initializes the Servlet using the init() method.

- **init(ServletConfig config) method** is called once in the Servlet's lifetime.
- It prepares the Servlet to handle requests (e.g., database connection setup).
- Called only once, unlike service() which is called per request.

Example:

```
@Override
public void init() throws ServletException {
    // Initialization code (e.g., database connection)
}
```

3. Handling request

Once initialized, the Servlet is ready to handle client requests.

3.1 Request and Response Objects

- ServletRequest and ServletResponse objects are created for each request.
- For HTTP, HttpServletRequest and HttpServletResponse objects are used.

3.2 The service() Method

- The container calls service(ServletRequest req, ServletResponse res) for each request.
- Determines the HTTP request type (GET, POST, PUT, DELETE).
- Delegates the request to appropriate methods: doGet(), doPost(), etc.

Example:

```
@Override
public void service(ServletRequest req, ServletResponse res)
        throws ServletException, IOException {
    // Handle request and generate response
```

}

4. Destroying a Servlet

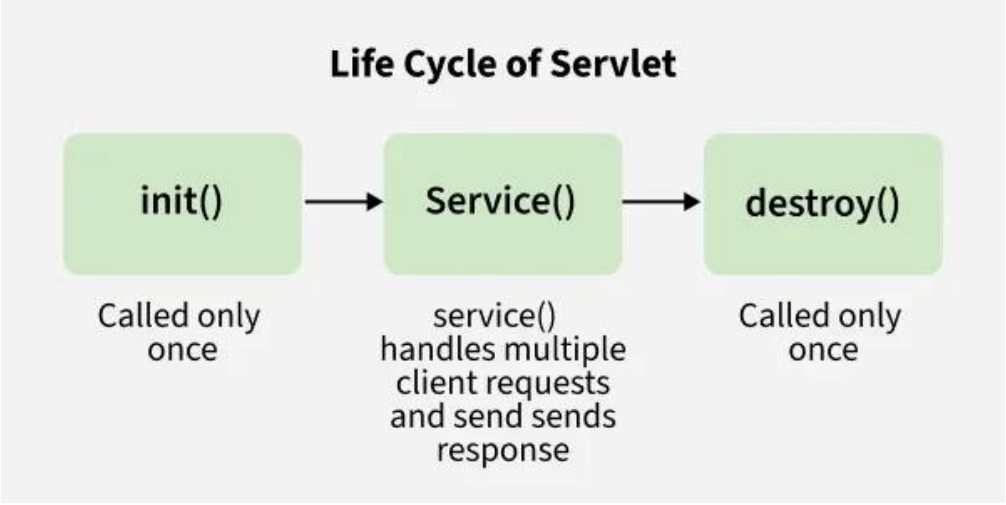
When the Servlet container decides to remove the Servlet, it follows these steps which are listed below

- **Allow Active Threads to Complete:** The container ensures that all threads executing the service() method complete their tasks.
- **Invoke the destroy() Method:** The container calls the destroy() method to allow the Servlet to release resources (e.g., closing database connections, freeing memory).
- **Release Servlet Instance:** After the destroy() method is executed, the Servlet container releases all references to the Servlet instance, making it eligible for garbage collection

Servlet Life Cycle Methods

There are three life cycle methods of a Servlet:

- init()
- service()
- destroy()



Life Cycle of Servlet

1. init() Method

- Invoked once when the Servlet is instantiated.
- Used to initialize resources (e.g., database connections).
- Prefer the non-parameterized init() method for fewer redundant calls.

Example:

```
@Override
public void init() throws ServletException {
    // Initialization code
}
```

- Exception handling is possible in init().
- Constructor is not recommended for initialization because it cannot throw ServletException.

2. service() Method

- Handles client requests and responses.
- Determines HTTP request type (GET, POST, PUT, DELETE) and delegates to doGet(), doPost(), etc.

Example:

```
@Override
public void service(ServletRequest req, ServletResponse res)
        throws ServletException, IOException {
    // Request handling code
}
```

- Connects client and server.
- Accepts ServletRequest and ServletResponse objects.

3. destroy() Method

- Called once at the end of the Servlet's life cycle.
- Cleans up resources (e.g., closes DB connections, releases memory).

Example:

```
@Override
public void destroy() {
   // Cleanup code
}
```

Java Servlet Example

Below is a sample program to illustrate Servlet in Java.

```
import jakarta.servlet.*;
import jakarta.servlet.http.*;
import java.io.*;


public class AdvanceJavaConcepts extends HttpServlet {
   private final String output; // Immutable (thread-safe)


   @Override
   public void init() throws ServletException {
      output = "Advance Java Concepts";
   }
```

```
    @Override
    protected void doGet(HttpServletRequest req, HttpServletResponse resp)
        throws ServletException, IOException {
        resp.setContentType("text/html");
        try (PrintWriter out = resp.getWriter()) {
            out.println(output); // Thread-safe
        }
    }

    @Override
    public void destroy() {
        System.out.println("Servlet destroyed");
    }
}
```

Explanation:

- **init() Method:** Initializes the output variable when the Servlet is first loaded.
- **doGet() Method:** Responds to client GET requests and prints the output as HTML.
- **Thread Safety:** output is immutable, ensuring safe access by multiple threads.
- **destroy() Method:** Invoked before Servlet removal to release resources or perform cleanup.

Servlets - Form Data

You must have come across many situations when you need to pass some information from your browser to web server and ultimately to your backend program. The browser uses two methods to pass this information to web server. These methods are GET Method and POST Method.

GET Method

The GET method sends the encoded user information appended to the page request. The page and the encoded information are separated by the **?** (question mark) symbol as follows −

http://www.test.com/hello?key1 = value1&key2 = value2

The GET method is the default method to pass information from browser to web server and it produces a long string that appears in your browser's Location:box. Never use the GET method if you have password or other sensitive information to pass to the server. The GET method has size limitation: only 1024 characters can be used in a request string.

This information is passed using QUERY_STRING header and will be accessible through QUERY_STRING environment variable and Servlet handles this type of requests using **doGet()** method.

POST Method

A generally more reliable method of passing information to a backend program is the POST method. This packages the information in exactly the same way as GET method, but instead of sending it as a text string after a ? (question mark) in the URL it sends it as a separate message. This message comes to the backend program in the form of the standard input which you can parse and use for your processing. Servlet handles this type of requests using **doPost()** method.

Reading Form Data using Servlet

Servlets handles form data parsing automatically using the following methods depending on the situation −

- **getParameter()** − You call request.getParameter() method to get the value of a form parameter.
- **getParameterValues()** − Call this method if the parameter appears more than once and returns multiple values, for example checkbox.
- **getParameterNames()** − Call this method if you want a complete list of all parameters in the current request.

GET Method Example using URL

Here is a simple URL which will pass two values to HelloForm program using GET method.

http://localhost:8080/HelloForm?first_name = ZARA&last_name = ALI

Given below is the **HelloForm.java** servlet program to handle input given by web browser. We are going to use **getParameter()** method which makes it very easy to access passed information −

```
// Import required java libraries

import java.io.*;

import javax.servlet.*;

import javax.servlet.http.*;


// Extend HttpServlet class

public class HelloForm extends HttpServlet {


  public void doGet(HttpServletRequest request, HttpServletResponse response)

    throws ServletException, IOException {


    // Set response content type
```

```
    response.setContentType("text/html");

    PrintWriter out = response.getWriter();
    String title = "Using GET Method to Read Form Data";
    String docType =
      "<!doctype html public \"-//w3c//dtd html 4.0 " + "transitional//en\">\n";

    out.println(docType +
      "<html>\n" +
        "<head><title>" + title + "</title></head>\n" +
        "<body bgcolor = \"#f0f0f0\">\n" +
          "<h1 align = \"center\">" + title + "</h1>\n" +
          "<ul>\n" +
            "  <li><b>First Name</b>: "
            + request.getParameter("first_name") + "\n" +
            "  <li><b>Last Name</b>: "
            + request.getParameter("last_name") + "\n" +
          "</ul>\n" +
        "</body>" +
      "</html>"
    );
  }
}
```

Assuming your environment is set up properly, compile HelloForm.java as follows –

$ javac HelloForm.java

If everything goes fine, above compilation would produce **HelloForm.class** file. Next you would have to copy this class file in <Tomcat-installationdirectory>/webapps/ROOT/WEB-INF/classes and create following entries in **web.xml** file located in <Tomcat-installation-directory>/webapps/ROOT/WEB-INF/

```
<servlet>
   <servlet-name>HelloForm</servlet-name>
   <servlet-class>HelloForm</servlet-class>
</servlet>

<servlet-mapping>
   <servlet-name>HelloForm</servlet-name>
   <url-pattern>/HelloForm</url-pattern>
</servlet-mapping>
```

Now type *http://localhost:8080/HelloForm?first_name=ZARA&last_name=ALI* in your browser's Location:box and make sure you already started tomcat server, before firing above command in the browser. This would generate following result –

Using GET Method to Read Form Data

- **First Name**: ZARA
- **Last Name**: ALI

GET Method Example Using Form

Here is a simple example which passes two values using HTML FORM and submit button. We are going to use same Servlet HelloForm to handle this input.

```
<html>
  <body>
    <form action = "HelloForm" method = "GET">
      First Name: <input type = "text" name = "first_name">
      <br />
      Last Name: <input type = "text" name = "last_name" />
      <input type = "submit" value = "Submit" />
    </form>
  </body>
</html>
```

Keep this HTML in a file Hello.htm and put it in <Tomcat-installationdirectory>/webapps/ROOT directory. When you would access *http://localhost:8080/Hello.htm*, here is the actual output of the above form.

First Name: Last Name:

Try to enter First Name and Last Name and then click submit button to see the result on your local machine where tomcat is running. Based on the input provided, it will generate similar result as mentioned in the above example.

POST Method Example Using Form

Let us do little modification in the above servlet, so that it can handle GET as well as POST methods. Below is **HelloForm.java** servlet program to handle input given by web browser using GET or POST methods.

```
// Import required java libraries
```

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

// Extend HttpServlet class
public class HelloForm extends HttpServlet {

  // Method to handle GET method request.
  public void doGet(HttpServletRequest request, HttpServletResponse response)
    throws ServletException, IOException {

    // Set response content type
    response.setContentType("text/html");

    PrintWriter out = response.getWriter();
    String title = "Using GET Method to Read Form Data";
    String docType =
      "<!doctype html public \"-//w3c//dtd html 4.0 " +
      "transitional//en\">\n";

    out.println(docType +
      "<html>\n" +
        "<head><title>" + title + "</title></head>\n" +
```

```
            "<body bgcolor = \"#f0f0f0\">\n" +
               "<h1 align = \"center\">" + title + "</h1>\n" +
               "<ul>\n" +
                  "  <li><b>First Name</b>: "
                  + request.getParameter("first_name") + "\n" +
                  "  <li><b>Last Name</b>: "
                  + request.getParameter("last_name") + "\n" +
               "</ul>\n" +
            "</body>"
         "</html>"
      );
   }

   // Method to handle POST method request.
   public void doPost(HttpServletRequest request, HttpServletResponse response)
      throws ServletException, IOException {

      doGet(request, response);
   }
}
```

Now compile and deploy the above Servlet and test it using Hello.htm with the POST method as follows −

```
<html>
```

```
  <body>
    <form action = "HelloForm" method = "POST">
      First Name: <input type = "text" name = "first_name">
      <br />
      Last Name: <input type = "text" name = "last_name" />
      <input type = "submit" value = "Submit" />
    </form>
  </body>
</html>
```

Here is the actual output of the above form, Try to enter First and Last Name and then click submit button to see the result on your local machine where tomcat is running.

First Name: Last Name:

Based on the input provided, it would generate similar result as mentioned in the above examples.

Passing Checkbox Data to Servlet Program

Checkboxes are used when more than one option is required to be selected.

Here is example HTML code, CheckBox.htm, for a form with two checkboxes

```
<html>
  <body>
    <form action = "CheckBox" method = "POST" target = "_blank">
      <input type = "checkbox" name = "maths" checked = "checked" /> Maths
      <input type = "checkbox" name = "physics"  /> Physics
      <input type = "checkbox" name = "chemistry" checked = "checked" />
```

```
                                    Chemistry

        <input type = "submit" value = "Select Subject" />

      </form>

   </body>

</html>
```

The result of this code is the following form

Maths Physics Chemistry

Given below is the CheckBox.java servlet program to handle input given by web browser for checkbox button.

```
// Import required java libraries

import java.io.*;

import javax.servlet.*;

import javax.servlet.http.*;


// Extend HttpServlet class

public class CheckBox extends HttpServlet {


   // Method to handle GET method request.

   public void doGet(HttpServletRequest request, HttpServletResponse response)

      throws ServletException, IOException {


      // Set response content type

      response.setContentType("text/html");
```

```
  PrintWriter out = response.getWriter();
  String title = "Reading Checkbox Data";
  String docType =
    "<!doctype html public \"-//w3c//dtd html 4.0 " + "transitional//en\">\n";

  out.println(docType +
    "<html>\n" +
      "<head><title>" + title + "</title></head>\n" +
      "<body bgcolor = \"#f0f0f0\">\n" +
        "<h1 align = \"center\">" + title + "</h1>\n" +
        "<ul>\n" +
          "  <li><b>Maths Flag : </b>: "
          + request.getParameter("maths") + "\n" +
          "  <li><b>Physics Flag: </b>: "
          + request.getParameter("physics") + "\n" +
          "  <li><b>Chemistry Flag: </b>: "
          + request.getParameter("chemistry") + "\n" +
        "</ul>\n" +
      "</body>"
    "</html>"
  );
}
```

```
  // Method to handle POST method request.
  public void doPost(HttpServletRequest request, HttpServletResponse response)
    throws ServletException, IOException {

    doGet(request, response);
  }
}
```

For the above example, it would display following result −

Reading Checkbox Data

- **Maths Flag :** : on
- **Physics Flag:** : null
- **Chemistry Flag:** : on

Reading All Form Parameters

Following is the generic example which uses **getParameterNames()** method of HttpServletRequest to read all the available form parameters. This method returns an Enumeration that contains the parameter names in an unspecified order

Once we have an Enumeration, we can loop down the Enumeration in standard way by, using *hasMoreElements()* method to determine when to stop and using *nextElement()* method to get each parameter name.

```
// Import required java libraries

import java.io.*;

import javax.servlet.*;
```

```
import javax.servlet.http.*;
import java.util.*;

// Extend HttpServlet class
public class ReadParams extends HttpServlet {

  // Method to handle GET method request.
  public void doGet(HttpServletRequest request, HttpServletResponse response)
    throws ServletException, IOException {

    // Set response content type
    response.setContentType("text/html");

    PrintWriter out = response.getWriter();
    String title = "Reading All Form Parameters";
    String docType =
      "<!doctype html public \"-//w3c//dtd html 4.0 " + "transitional//en\">\n";

    out.println(docType +
      "<html>\n" +
      "<head><title>" + title + "</title></head>\n" +
      "<body bgcolor = \"#f0f0f0\">\n" +
      "<h1 align = \"center\">" + title + "</h1>\n" +
```

```
  "<table width = \"100%\" border = \"1\" align = \"center\">\n" +
  "<tr bgcolor = \"#949494\">\n" +
    "<th>Param Name</th>"
    "<th>Param Value(s)</th>\n"+
  "</tr>\n"
);

Enumeration paramNames = request.getParameterNames();

while(paramNames.hasMoreElements()) {
  String paramName = (String)paramNames.nextElement();
  out.print("<tr><td>" + paramName + "</td>\n<td>");
  String[] paramValues = request.getParameterValues(paramName);

  // Read single valued data
  if (paramValues.length == 1) {
    String paramValue = paramValues[0];
    if (paramValue.length() == 0)
      out.println("<i>No Value</i>");
      else
      out.println(paramValue);
  } else {
    // Read multiple valued data
```

```
            out.println("<ul>");

            for(int i = 0; i < paramValues.length; i++) {
               out.println("<li>" + paramValues[i]);
            }
            out.println("</ul>");
         }
      }
      out.println("</tr>\n</table>\n</body></html>");
   }

   // Method to handle POST method request.
   public void doPost(HttpServletRequest request, HttpServletResponse response)
      throws ServletException, IOException {

      doGet(request, response);
   }
}
```

Now, try the above servlet with the following form −

```
<html>
   <body>
      <form action = "ReadParams" method = "POST" target = "_blank">
         <input type = "checkbox" name = "maths" checked = "checked" /> Maths
```

```
      <input type = "checkbox" name = "physics"  /> Physics
      <input type = "checkbox" name = "chemistry" checked = "checked" /> Chem
      <input type = "submit" value = "Select Subject" />
   </form>
  </body>
</html>
```

Now calling servlet using the above form would generate the following result −

Reading All Form Parameters

Param Name	Param Value(s)
maths	on
chemistry	on

You can try the above servlet to read any other form's data having other objects like text box, radio button or drop down box etc.

Servlets - Client HTTP Request

When a browser requests for a web page, it sends lot of information to the web server which cannot be read directly because this information travel as a part of header of HTTP request. You can check HTTP Protocol for more information on this.

Following is the important header information which comes from browser side and you would use very frequently in web programming −

Sr.No.	Header & Description
1	**Accept** This header specifies the MIME types that the browser or other clients can handle. Values of **image/png** or **image/jpeg** are the two most common possibilities.
2	**Accept-Charset** This header specifies the character sets the browser can use to display the information. For example ISO-8859-1.
3	**Accept-Encoding** This header specifies the types of encodings that the browser knows how to handle. Values of **gzip** or **compress** are the two most common possibilities.
4	**Accept-Language** This header specifies the client's preferred languages in case the servlet can produce results in more than one language. For example en, en-us, ru, etc
5	**Authorization**

	This header is used by clients to identify themselves when accessing password-protected Web pages.
6	**Connection** This header indicates whether the client can handle persistent HTTP connections. Persistent connections permit the client or other browser to retrieve multiple files with a single request. A value of **Keep-Alive** means that persistent connections should be used.
7	**Content-Length** This header is applicable only to POST requests and gives the size of the POST data in bytes.
8	**Cookie** This header returns cookies to servers that previously sent them to the browser.
9	**Host** This header specifies the host and port as given in the original URL.
10	**If-Modified-Since** This header indicates that the client wants the page only if it has been changed after the specified date. The server sends a code, 304 which means **Not Modified** header if no newer result is available.
11	**If-Unmodified-Since** This header is the reverse of If-Modified-Since; it specifies that the operation should succeed only if the document is older than the specified date.

12	**Referer** This header indicates the URL of the referring Web page. For example, if you are at Web page 1 and click on a link to Web page 2, the URL of Web page 1 is included in the Referrer header when the browser requests Web page 2.
13	**User-Agent** This header identifies the browser or other client making the request and can be used to return different content to different types of browsers.

Methods to read HTTP Header

There are following methods which can be used to read HTTP header in your servlet program. These methods are available with *HttpServletRequest* object

Sr.No.	Method & Description
1	**Cookie[] getCookies()** Returns an array containing all of the Cookie objects the client sent with this request.
2	**Enumeration getAttributeNames()** Returns an Enumeration containing the names of the attributes available to this request.
3	**Enumeration getHeaderNames()** Returns an enumeration of all the header names this request contains.
4	**Enumeration getParameterNames()**

	Returns an Enumeration of String objects containing the names of the parameters contained in this request
5	**HttpSession getSession()** Returns the current session associated with this request, or if the request does not have a session, creates one.
6	**HttpSession getSession(boolean create)** Returns the current HttpSession associated with this request or, if if there is no current session and value of create is true, returns a new session.
7	**Locale getLocale()** Returns the preferred Locale that the client will accept content in, based on the Accept-Language header.
8	**Object getAttribute(String name)** Returns the value of the named attribute as an Object, or null if no attribute of the given name exists.
9	**ServletInputStream getInputStream()** Retrieves the body of the request as binary data using a ServletInputStream.
10	**String getAuthType()** Returns the name of the authentication scheme used to protect the servlet, for example, "BASIC" or "SSL," or null if the JSP was not protected.
11	**String getCharacterEncoding()** Returns the name of the character encoding used in the body of this request.

12	**String getContentType()** Returns the MIME type of the body of the request, or null if the type is not known.
13	**String getContextPath()** Returns the portion of the request URI that indicates the context of the request.
14	**String getHeader(String name)** Returns the value of the specified request header as a String.
15	**String getMethod()** Returns the name of the HTTP method with which this request was made, for example, GET, POST, or PUT.
16	**String getParameter(String name)** Returns the value of a request parameter as a String, or null if the parameter does not exist.
17	**String getPathInfo()** Returns any extra path information associated with the URL the client sent when it made this request
18	**String getProtocol()** Returns the name and version of the protocol the request.
19	**String getQueryString()**

	Returns the query string that is contained in the request URL after the path.
20	**String getRemoteAddr()** Returns the Internet Protocol (IP) address of the client that sent the request.
21	**String getRemoteHost()** Returns the fully qualified name of the client that sent the request.
22	**String getRemoteUser()** Returns the login of the user making this request, if the user has been authenticated, or null if the user has not been authenticated.
23	**String getRequestURI()** Returns the part of this request's URL from the protocol name up to the query string in the first line of the HTTP request.
24	**String getRequestedSessionId()** Returns the session ID specified by the client.
25	**String getServletPath()** Returns the part of this request's URL that calls the JSP.
26	**String[] getParameterValues(String name)** Returns an array of String objects containing all of the values the given request parameter has, or null if the parameter does not exist.
27	**boolean isSecure()**

	Returns a Boolean indicating whether this request was made using a secure channel, such as HTTPS.
28	**int getContentLength()** Returns the length, in bytes, of the request body and made available by the input stream, or -1 if the length is not known.
29	**int getIntHeader(String name)** Returns the value of the specified request header as an int.
30	**int getServerPort()** Returns the port number on which this request was received.

HTTP Header Request Example

Following is the example which uses **getHeaderNames()** method of HttpServletRequest to read the HTTP header information. This method returns an Enumeration that contains the header information associated with the current HTTP request.

Once we have an Enumeration, we can loop down the Enumeration in the standard manner, using *hasMoreElements()* method to determine when to stop and using *nextElement()* method to get each parameter name

```
// Import required java libraries

import java.io.*;

import javax.servlet.*;

import javax.servlet.http.*;

import java.util.*;
```

```
// Extend HttpServlet class
public class DisplayHeader extends HttpServlet {

  // Method to handle GET method request.
  public void doGet(HttpServletRequest request, HttpServletResponse response)
    throws ServletException, IOException {

    // Set response content type
    response.setContentType("text/html");

    PrintWriter out = response.getWriter();
    String title = "HTTP Header Request Example";
    String docType =
      "<!doctype html public \"-//w3c//dtd html 4.0 " + "transitional//en\">\n";

    out.println(docType +
      "<html>\n" +
      "<head><title>" + title + "</title></head>\n"+
      "<body bgcolor = \"#f0f0f0\">\n" +
      "<h1 align = \"center\">" + title + "</h1>\n" +
      "<table width = \"100%\" border = \"1\" align = \"center\">\n" +
      "<tr bgcolor = \"#949494\">\n" +
      "<th>Header Name</th><th>Header Value(s)</th>\n"+
```

```
        "</tr>\n"
      );

      Enumeration headerNames = request.getHeaderNames();

      while(headerNames.hasMoreElements()) {
        String paramName = (String)headerNames.nextElement();
        out.print("<tr><td>" + paramName + "</td>\n");
        String paramValue = request.getHeader(paramName);
        out.println("<td> " + paramValue + "</td></tr>\n");
      }
      out.println("</table>\n</body></html>");
    }

    // Method to handle POST method request.
    public void doPost(HttpServletRequest request, HttpServletResponse response)
      throws ServletException, IOException {

      doGet(request, response);
    }
}
```

Now calling the above servlet would generate the following result −

HTTP Header Request Example

Header Name	Header Value(s)
accept	*/*
accept-language	en-us
user-agent	Mozilla/4.0 (compatible; MSIE 7.0; Windows NT 5.1; Trident/4.0; InfoPath.2; MS-RTC LM 8)
accept-encoding	gzip, deflate
host	localhost:8080
connection	Keep-Alive
cache-control	no-cache

Servlets - Server HTTP Response

As discussed in the previous chapter, when a Web server responds to an HTTP request, the response typically consists of a status line, some response headers, a blank line, and the document. A typical response looks like this −

HTTP/1.1 200 OK

Content-Type: text/html

Header2: ...

...

HeaderN: ...

(Blank Line)

<!doctype ...>

<html>

<head>...</head>

<body>

...

</body>

</html>

The status line consists of the HTTP version (HTTP/1.1 in the example), a status code (200 in the example), and a very short message corresponding to the status code (OK in the example).

Following is a summary of the most useful HTTP 1.1 response headers which go back to the browser from web server side and you would use them very frequently in web programming –

Sr.No.	Header &amp Description
1	**Allow** This header specifies the request methods (GET, POST, etc.) that the server supports.
2	**Cache-Control** This header specifies the circumstances in which the response document can safely be cached. It can have values **public**, **private** or **no-cache** etc. Public means document is cacheable, Private means document is for a single user and can only be stored in private (non-shared) caches and nocache means document should never be cached.
3	**Connection** This header instructs the browser whether to use persistent in HTTP connections or not. A value of **close** instructs the browser not to use persistent HTTP connections and **keepalive** means using persistent connections.
4	**Content-Disposition** This header lets you request that the browser ask the user to save the response to disk in a file of the given name.
5	**Content-Encoding** This header specifies the way in which the page was encoded during transmission.
6	**Content-Language**

	This header signifies the language in which the document is written. For example en, en-us, ru, etc
7	**Content-Length** This header indicates the number of bytes in the response. This information is needed only if the browser is using a persistent (keep-alive) HTTP connection.
8	**Content-Type** This header gives the MIME (Multipurpose Internet Mail Extension) type of the response document.
9	**Expires** This header specifies the time at which the content should be considered out-of-date and thus no longer be cached.
10	**Last-Modified** This header indicates when the document was last changed. The client can then cache the document and supply a date by an **If-Modified-Since** request header in later requests.
11	**Location** This header should be included with all responses that have a status code in the 300s. This notifies the browser of the document address. The browser automatically reconnects to this location and retrieves the new document.
12	**Refresh**

	This header specifies how soon the browser should ask for an updated page. You can specify time in number of seconds after which a page would be refreshed.
13	**Retry-After** This header can be used in conjunction with a 503 (Service Unavailable) response to tell the client how soon it can repeat its request.
14	**Set-Cookie** This header specifies a cookie associated with the page.

Methods to Set HTTP Response Header

There are following methods which can be used to set HTTP response header in your servlet program. These methods are available with *HttpServletResponse* object.

Sr.No.	Method & Description
1	**String encodeRedirectURL(String url)** Encodes the specified URL for use in the sendRedirect method or, if encoding is not needed, returns the URL unchanged.
2	**String encodeURL(String url)** Encodes the specified URL by including the session ID in it, or, if encoding is not needed, returns the URL unchanged.
3	**boolean containsHeader(String name)** Returns a Boolean indicating whether the named response header has already been set.

4	**boolean isCommitted()** Returns a Boolean indicating if the response has been committed.
5	**void addCookie(Cookie cookie)** Adds the specified cookie to the response.
6	**void addDateHeader(String name, long date)** Adds a response header with the given name and date-value.
7	**void addHeader(String name, String value)** Adds a response header with the given name and value.
8	**void addIntHeader(String name, int value)** Adds a response header with the given name and integer value.
9	**void flushBuffer()** Forces any content in the buffer to be written to the client.
10	**void reset()** Clears any data that exists in the buffer as well as the status code and headers.
11	**void resetBuffer()** Clears the content of the underlying buffer in the response without clearing headers or status code.
12	**void sendError(int sc)**

	Sends an error response to the client using the specified status code and clearing the buffer.
13	**void sendError(int sc, String msg)** Sends an error response to the client using the specified status.
14	**void sendRedirect(String location)** Sends a temporary redirect response to the client using the specified redirect location URL.
15	**void setBufferSize(int size)** Sets the preferred buffer size for the body of the response.
16	**void setCharacterEncoding(String charset)** Sets the character encoding (MIME charset) of the response being sent to the client, for example, to UTF-8.
17	**void setContentLength(int len)** Sets the length of the content body in the response In HTTP servlets, this method sets the HTTP Content-Length header.
18	**void setContentType(String type)** Sets the content type of the response being sent to the client, if the response has not been committed yet.
19	**void setDateHeader(String name, long date)** Sets a response header with the given name and date-value.

20	**void setHeader(String name, String value)** Sets a response header with the given name and value.
21	**void setIntHeader(String name, int value)** Sets a response header with the given name and integer value
22	**void setLocale(Locale loc)** Sets the locale of the response, if the response has not been committed yet.
23	**void setStatus(int sc)** Sets the status code for this response

HTTP Header Response Example

You already have seen setContentType() method working in previous examples and following example would also use same method, additionally we would use **setIntHeader()** method to set **Refresh** header.

```
// Import required java libraries

import java.io.*;

import javax.servlet.*;

import javax.servlet.http.*;

import java.util.*;


// Extend HttpServlet class

public class Refresh extends HttpServlet {


  // Method to handle GET method request.
```

```
public void doGet(HttpServletRequest request, HttpServletResponse response)
  throws ServletException, IOException {

  // Set refresh, autoload time as 5 seconds
  response.setIntHeader("Refresh", 5);

  // Set response content type
  response.setContentType("text/html");

  // Get current time
  Calendar calendar = new GregorianCalendar();
  String am_pm;
  int hour = calendar.get(Calendar.HOUR);
  int minute = calendar.get(Calendar.MINUTE);
  int second = calendar.get(Calendar.SECOND);

  if(calendar.get(Calendar.AM_PM) == 0)
    am_pm = "AM";
  else
    am_pm = "PM";

  String CT = hour+":"+ minute +":"+ second +" "+ am_pm;
```

```
      PrintWriter out = response.getWriter();
      String title = "Auto Refresh Header Setting";
      String docType =
        "<!doctype html public \"-//w3c//dtd html 4.0 " + "transitional//en\">\n";

      out.println(docType +
        "<html>\n" +
        "<head><title>" + title + "</title></head>\n"+
        "<body bgcolor = \"#f0f0f0\">\n" +
        "<h1 align = \"center\">" + title + "</h1>\n" +
        "<p>Current Time is: " + CT + "</p>\n"
      );
   }

   // Method to handle POST method request.
   public void doPost(HttpServletRequest request, HttpServletResponse response)
      throws ServletException, IOException {

      doGet(request, response);
   }
}
```

Now calling the above servlet would display current system time after every 5 seconds as follows. Just run the servlet and wait to see the result −

Auto Refresh Header Setting

Current Time is: 9:44:50 PM

HTTP - Status Codes

The Status-Code element in a server response, is a 3-digit integer where the first digit of the Status-Code defines the class of response and the last two digits do not have any categorization role. There are 5 values for the first digit:

Status Code	Description
1xx: Informational	It means the request has been received and the process is continuing.
2xx: Success	It means the action was successfully received, understood, and accepted.
3xx: Redirection	It means further action must be taken in order to complete the request.
4xx: Client Error	It means the request contains incorrect syntax or cannot be fulfilled.
5xx: Server Error	It means the server failed to fulfill an apparently valid request.

HTTP status codes are extensible and HTTP applications are not required to understand the meaning of all the registered status codes. Given below is a list of all the status codes.

1xx: Information

Message	Description
100 Continue	Only a part of the request has been received by the server, but as long as it has not been rejected, the client should continue with the request.

101 Switching Protocols	The server switches protocol.

2xx: Successful

Message	Description
200 OK	The request is OK.
201 Created	The request is complete, and a new resource is created.
202 Accepted	The request is accepted for processing, but the processing is not complete.
203 Non-authoritative Information	The information in the entity header is from a local or third-party copy, not from the original server.
204 No Content	A status code and a header are given in the response, but there is no entity-body in the reply.
205 Reset Content	The browser should clear the form used for this transaction for additional input.
206 Partial Content	The server is returning partial data of the size requested. Used in response to a request specifying a *Range* header. The server must specify the range included in the response with the *Content-Range* header.

Example

GET /index.htm HTTP/1.1

User-Agent: Mozilla/4.0

Host: www.tutorialspoint.com

Accept-Language: en-us

Accept-Encoding: gzip, deflate

Connection: Keep-Alive

The server response against the above GET request will be as follows:

HTTP/1.1 200 OK

Date: Tue, 16 Dec 2024 18:59:38 GMT

Server: Apache/2.4.59 (Ubuntu)

Vary: Accept-Encoding,Origin

Content-Length: 20241

Content-Type: text/html; charset=UTF-8

3xx: Redirection

Message	Description
300 Multiple Choices	A link list. The user can select a link and go to that location. Maximum five addresses.
301 Moved Permanently	The requested page has moved to a new url.
302 Found	The requested page has moved temporarily to a new url.
303 See Other	The requested page can be found under a different url.

304 Not Modified	This is the response code to an *If-Modified-Since* or *If-None-Match* header, where the URL has not been modified since the specified date.
305 Use Proxy	The requested URL must be accessed through the proxy mentioned in the *Location* header.
306 *Unused*	This code was used in a previous version. It is no longer used, but the code is reserved.
307 Temporary Redirect	The requested page has moved temporarily to a new url.

Example

GET httpbin.org/redirect/1 HTTP/1.1

The server response against the above GET request will be as follows:

HTTP/1.1 302 FOUND

Date: Tue, 17 Dec 2024 07:55:56 GMT

Content-Type: text/html; charset=utf-8

Content-Length: 215

Server: gunicorn/19.9.0

4xx: Client Error

Message	Description
400Bad Request	The server did not understand the request.
401 Unauthorized	The requested page needs a username and a password.

402 Payment Required	*You can not use this code yet*.
403 Forbidden	Access is forbidden to the requested page.
404 Not Found	The server can not find the requested page.
405 Method Not Allowed	The method specified in the request is not allowed.
406 Not Acceptable	The server can only generate a response that is not accepted by the client.
407 Proxy Authentication Required	You must authenticate with a proxy server before this request can be served.
408 Request Timeout	The request took longer than the server was prepared to wait.
409 Conflict	The request could not be completed because of a conflict.
410 Gone	The requested page is no longer available.
411 Length Required	The "Content-Length" is not defined. The server will not accept the request without it.
412 Precondition Failed	The pre condition given in the request evaluated to false by the server.

413 Request Entity Too Large	The server will not accept the request, because the request entity is too large.
414 Request-url Too Long	The server will not accept the request, because the url is too long. Occurs when you convert a "post" request to a "get" request with a long query information.
415 Unsupported Media Type	The server will not accept the request, because the mediatype is not supported.
416 Requested Range Not Satisfiable	The requested byte range is not available and is out of bounds.
417 Expectation Failed	The expectation given in an Expect request-header field could not be met by this server.

Example

GET httpbin.org/post HTTP/1.1

The server response against the above GET request will be as follows:

HTTP/1.1 405 METHOD NOT ALLOWED

Date: Tue, 17 Dec 2024 07:58:43 GMT

Content-Type: text/html

Content-Length: 178

Server: gunicorn/19.9.0

Allow: POST, OPTIONS

5xx: Server Error

Message	Description

500Internal Server Error	The request was not completed. The server met an unexpected condition.
501 Not Implemented	The request was not completed. The server did not support the functionality required.
502 Bad Gateway	The request was not completed. The server received an invalid response from the upstream server.
503 Service Unavailable	The request was not completed. The server is temporarily overloading or down.
504 Gateway Timeout	The gateway has timed out.
505 HTTP Version Not Supported	The server does not support the "http protocol" version.

Example

GET httpbin.org/status/500 HTTP/1.1

The server response against the above GET request will be as follows:

HTTP/1.1 500 INTERNAL SERVER ERROR

Date: Tue, 17 Dec 2024 08:05:12 GMT

Content-Type: text/html; charset=utf-8

Content-Length: 0

Server: gunicorn/19.9.0

Servlets - Cookies Handling

Cookies are text files stored on the client computer and they are kept for various information tracking purpose. Java Servlets transparently supports HTTP cookies.

There are three steps involved in identifying returning users −

- Server script sends a set of cookies to the browser. For example name, age, or identification number etc.
- Browser stores this information on local machine for future use.
- When next time browser sends any request to web server then it sends those cookies information to the server and server uses that information to identify the user.

This chapter will teach you how to set or reset cookies, how to access them and how to delete them.

The Anatomy of a Cookie

Cookies are usually set in an HTTP header (although JavaScript can also set a cookie directly on a browser). A servlet that sets a cookie might send headers that look something like this −

HTTP/1.1 200 OK

Date: Fri, 04 Feb 2000 21:03:38 GMT

Server: Apache/1.3.9 (UNIX) PHP/4.0b3

Set-Cookie: name = xyz; expires = Friday, 04-Feb-07 22:03:38 GMT;

path = /; domain = tutorialspoint.com

Connection: close

Content-Type: text/html

As you can see, the Set-Cookie header contains a name value pair, a GMT date, a path and a domain. The name and value will be URL encoded. The expires field is an instruction to the browser to "forget" the cookie after the given time and date.

If the browser is configured to store cookies, it will then keep this information until the expiry date. If the user points the browser at any page that matches the path and domain of the cookie, it will resend the cookie to the server. The browser's headers might look something like this −

GET / HTTP/1.0

Connection: Keep-Alive

User-Agent: Mozilla/4.6 (X11; I; Linux 2.2.6-15apmac ppc)

Host: zink.demon.co.uk:1126

Accept: image/gif, */*

Accept-Encoding: gzip

Accept-Language: en

Accept-Charset: iso-8859-1,*,utf-8

Cookie: name = xyz

A servlet will then have access to the cookie through the request method *request.getCookies()* which returns an array of *Cookie* objects.

Servlet Cookies Methods

Following is the list of useful methods which you can use while manipulating cookies in servlet.

Sr.No.	**Method & Description**
1	**public void setDomain(String pattern)** This method sets the domain to which cookie applies, for example tutorialspoint.com.
2	**public String getDomain()**

	This method gets the domain to which cookie applies, for example tutorialspoint.com.
3	**public void setMaxAge(int expiry)** This method sets how much time (in seconds) should elapse before the cookie expires. If you don't set this, the cookie will last only for the current session.
4	**public int getMaxAge()** This method returns the maximum age of the cookie, specified in seconds, By default, -1 indicating the cookie will persist until browser shutdown.
5	**public String getName()** This method returns the name of the cookie. The name cannot be changed after creation.
6	**public void setValue(String newValue)** This method sets the value associated with the cookie
7	**public String getValue()** This method gets the value associated with the cookie.
8	**public void setPath(String uri)** This method sets the path to which this cookie applies. If you don't specify a path, the cookie is returned for all URLs in the same directory as the current page as well as all subdirectories.
9	**public String getPath()**

	This method gets the path to which this cookie applies.
10	**public void setSecure(boolean flag)** This method sets the boolean value indicating whether the cookie should only be sent over encrypted (i.e. SSL) connections.
11	**public void setComment(String purpose)** This method specifies a comment that describes a cookie's purpose. The comment is useful if the browser presents the cookie to the user.
12	**public String getComment()** This method returns the comment describing the purpose of this cookie, or null if the cookie has no comment.

Setting Cookies with Servlet

Setting cookies with servlet involves three steps −

(1) Creating a Cookie object − You call the Cookie constructor with a cookie name and a cookie value, both of which are strings.

Cookie cookie = new Cookie("key","value");

Keep in mind, neither the name nor the value should contain white space or any of the following characters −

[ ] ( ) = , " / ? @ : ;

(2) Setting the maximum age − You use setMaxAge to specify how long (in seconds) the cookie should be valid. Following would set up a cookie for 24 hours.

cookie.setMaxAge(60 * 60 * 24);

(3) Sending the Cookie into the HTTP response headers − You use response.addCookie to add cookies in the HTTP response header as follows −

response.addCookie(cookie);

Example

Let us modify our Form Example to set the cookies for first and last name.

```
// Import required java libraries
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

// Extend HttpServlet class
public class HelloForm extends HttpServlet {

  public void doGet(HttpServletRequest request, HttpServletResponse response)
    throws ServletException, IOException {

    // Create cookies for first and last names.
    Cookie firstName = new Cookie("first_name",
request.getParameter("first_name"));
    Cookie lastName = new Cookie("last_name",
request.getParameter("last_name"));

    // Set expiry date after 24 Hrs for both the cookies.
    firstName.setMaxAge(60*60*24);
    lastName.setMaxAge(60*60*24);
```

```
// Add both the cookies in the response header.
response.addCookie( firstName );
response.addCookie( lastName );

// Set response content type
response.setContentType("text/html");

PrintWriter out = response.getWriter();
String title = "Setting Cookies Example";
String docType =
  "<!doctype html public \"-//w3c//dtd html 4.0 " + "transitional//en\">\n";

out.println(docType +
  "<html>\n" +
    "<head>
      <title>" + title + "</title>
    </head>\n" +

    "<body bgcolor = \"#f0f0f0\">\n" +
      "<h1 align = \"center\">" + title + "</h1>\n" +
      "<ul>\n" +
        "  <li><b>First Name</b>: "
```

```
            + request.getParameter("first_name") + "\n" +
            "  <li><b>Last Name</b>: "
            + request.getParameter("last_name") + "\n" +
          "</ul>\n" +
        "</body>
      </html>"
    );
  }
}
```

Compile the above servlet **HelloForm** and create appropriate entry in web.xml file and finally try following HTML page to call servlet.

```
<html>
  <body>
    <form action = "HelloForm" method = "GET">
      First Name: <input type = "text" name = "first_name">
      <br />
      Last Name: <input type = "text" name = "last_name" />
      <input type = "submit" value = "Submit" />
    </form>
  </body>
</html>
```

Keep above HTML content in a file Hello.htm and put it in <Tomcat-installationdirectory>/webapps/ROOT directory. When you would access *http://localhost:8080/Hello.htm*, here is the actual output of the above form.

First Name:
Last Name:

Try to enter First Name and Last Name and then click submit button. This would display first name and last name on your screen and same time it would set two cookies firstName and lastName which would be passed back to the server when next time you would press Submit button.

Next section would explain you how you would access these cookies back in your web application.

Reading Cookies with Servlet

To read cookies, you need to create an array of *javax.servlet.http.Cookie* objects by calling the **getCookies()** method of *HttpServletRequest*. Then cycle through the array, and use getName() and getValue() methods to access each cookie and associated value.

Example

Let us read cookies which we have set in previous example −

```
// Import required java libraries

import java.io.*;

import javax.servlet.*;

import javax.servlet.http.*;


// Extend HttpServlet class

public class ReadCookies extends HttpServlet {
```

```
public void doGet(HttpServletRequest request, HttpServletResponse response)
  throws ServletException, IOException {

  Cookie cookie = null;
  Cookie[] cookies = null;

  // Get an array of Cookies associated with this domain
  cookies = request.getCookies();

  // Set response content type
  response.setContentType("text/html");

  PrintWriter out = response.getWriter();
  String title = "Reading Cookies Example";
  String docType =
    "<!doctype html public \"-//w3c//dtd html 4.0 " +
    "transitional//en\">\n";

  out.println(docType +
    "<html>\n" +
    "<head><title>" + title + "</title></head>\n" +
    "<body bgcolor = \"#f0f0f0\">\n" );
```

```
      if( cookies != null ) {
         out.println("<h2> Found Cookies Name and Value</h2>");

         for (int i = 0; i < cookies.length; i++) {
            cookie = cookies[i];
            out.print("Name : " + cookie.getName( ) + ",  ");
            out.print("Value: " + cookie.getValue( ) + " <br/>");
         }
      } else {
         out.println("<h2>No cookies founds</h2>");
      }
      out.println("</body>");
      out.println("</html>");
   }
}
```

Compile above servlet **ReadCookies** and create appropriate entry in web.xml file. If you would have set first_name cookie as "John" and last_name cookie as "Player" then running *http://localhost:8080/ReadCookies* would display the following result −

Found Cookies Name and Value

Name : first_name, Value: John

Name : last_name, Value: Player

Delete Cookies with Servlet

To delete cookies is very simple. If you want to delete a cookie then you simply need to follow up following three steps −

- Read an already existing cookie and store it in Cookie object.
- Set cookie age as zero using **setMaxAge()** method to delete an existing cookie
- Add this cookie back into response header.

Example

The following example would delete and existing cookie named "first_name" and when you would run ReadCookies servlet next time it would return null value for first_name.

```
// Import required java libraries

import java.io.*;

import javax.servlet.*;

import javax.servlet.http.*;


// Extend HttpServlet class

public class DeleteCookies extends HttpServlet {


  public void doGet(HttpServletRequest request, HttpServletResponse response)

    throws ServletException, IOException {


    Cookie cookie = null;

    Cookie[] cookies = null;
```

```
// Get an array of Cookies associated with this domain
cookies = request.getCookies();

// Set response content type
response.setContentType("text/html");

PrintWriter out = response.getWriter();
String title = "Delete Cookies Example";
String docType =
  "<!doctype html public \"-//w3c//dtd html 4.0 " + "transitional//en\">\n";

out.println(docType +
  "<html>\n" +
  "<head><title>" + title + "</title></head>\n" +
  "<body bgcolor = \"#f0f0f0\">\n" );

if( cookies != null ) {
  out.println("<h2> Cookies Name and Value</h2>");

  for (int i = 0; i < cookies.length; i++) {
    cookie = cookies[i];
```

```
         if((cookie.getName( )).compareTo("first_name") == 0 ) {
            cookie.setMaxAge(0);
            response.addCookie(cookie);
            out.print("Deleted cookie : " + cookie.getName( ) + "<br/>");
         }
         out.print("Name : " + cookie.getName( ) + ",  ");
         out.print("Value: " + cookie.getValue( )+" <br/>");
      }
   } else {
      out.println("<h2>No cookies founds</h2>");
   }
   out.println("</body>");
   out.println("</html>");
  }
}
```

Compile above servlet **DeleteCookies** and create appropriate entry in web.xml file. Now running *http://localhost:8080/DeleteCookies* would display the following result −

Cookies Name and Value

Deleted cookie : first_name

Name : first_name, Value: John

Name : last_name, Value: Player

Now try to run *http://localhost:8080/ReadCookies* and it would display only one cookie as follows −

Found Cookies Name and Value

Name : last_name, Value: Player

You can delete your cookies in Internet Explorer manually. Start at the Tools menu and select Internet Options. To delete all cookies, press Delete Cookies.

Servlets - Session Tracking

HTTP is a "stateless" protocol which means each time a client retrieves a Web page, the client opens a separate connection to the Web server and the server automatically does not keep any record of previous client request.

Still there are following three ways to maintain session between web client and web server −

Cookies

A webserver can assign a unique session ID as a cookie to each web client and for subsequent requests from the client they can be recognized using the recieved cookie.

This may not be an effective way because many time browser does not support a cookie, so I would not recommend to use this procedure to maintain the sessions.

Hidden Form Fields

A web server can send a hidden HTML form field along with a unique session ID as follows −

<input type = "hidden" name = "sessionid" value = "12345">

This entry means that, when the form is submitted, the specified name and value are automatically included in the GET or POST data. Each time when web browser sends request back, then session_id value can be used to keep the track of different web browsers.

This could be an effective way of keeping track of the session but clicking on a regular (<A HREF...>) hypertext link does not result in a form submission, so hidden form fields also cannot support general session tracking.

URL Rewriting

You can append some extra data on the end of each URL that identifies the session, and the server can associate that session identifier with data it has stored about that session.

For example, with http://tutorialspoint.com/file.htm;sessionid = 12345, the session identifier is attached as sessionid = 12345 which can be accessed at the web server to identify the client.

URL rewriting is a better way to maintain sessions and it works even when browsers don't support cookies. The drawback of URL re-writing is that you would have to generate every URL dynamically to assign a session ID, even in case of a simple static HTML page.

The HttpSession Object

Apart from the above mentioned three ways, servlet provides HttpSession Interface which provides a way to identify a user across more than one page request or visit to a Web site and to store information about that user.

The servlet container uses this interface to create a session between an HTTP client and an HTTP server. The session persists for a specified time period, across more than one connection or page request from the user.

You would get HttpSession object by calling the public method **getSession()** of HttpServletRequest, as below −

HttpSession session = request.getSession();

You need to call *request.getSession()* before you send any document content to the client. Here is a summary of the important methods available through HttpSession object −

Sr.No.	Method & Description
1	**public Object getAttribute(String name)** This method returns the object bound with the specified name in this session, or null if no object is bound under the name.
2	**public Enumeration getAttributeNames()**

	This method returns an Enumeration of String objects containing the names of all the objects bound to this session.
3	**public long getCreationTime()** This method returns the time when this session was created, measured in milliseconds since midnight January 1, 1970 GMT.
4	**public String getId()** This method returns a string containing the unique identifier assigned to this session.
5	**public long getLastAccessedTime()** This method returns the last accessed time of the session, in the format of milliseconds since midnight January 1, 1970 GMT
6	**public int getMaxInactiveInterval()** This method returns the maximum time interval (seconds), that the servlet container will keep the session open between client accesses.
7	**public void invalidate()** This method invalidates this session and unbinds any objects bound to it.
8	**public boolean isNew(** This method returns true if the client does not yet know about the session or if the client chooses not to join the session.
9	**public void removeAttribute(String name)**

	This method removes the object bound with the specified name from this session.
10	**public void setAttribute(String name, Object value)** This method binds an object to this session, using the name specified.
11	**public void setMaxInactiveInterval(int interval)** This method specifies the time, in seconds, between client requests before the servlet container will invalidate this session.

Session Tracking Example

This example describes how to use the HttpSession object to find out the creation time and the last-accessed time for a session. We would associate a new session with the request if one does not already exist.

```
// Import required java libraries

import java.io.*;

import javax.servlet.*;

import javax.servlet.http.*;

import java.util.*;


// Extend HttpServlet class

public class SessionTrack extends HttpServlet {


  public void doGet(HttpServletRequest request, HttpServletResponse response)

    throws ServletException, IOException {
```

```
// Create a session object if it is already not  created.
HttpSession session = request.getSession(true);

// Get session creation time.
Date createTime = new Date(session.getCreationTime());

// Get last access time of this web page.
Date lastAccessTime = new Date(session.getLastAccessedTime());

String title = "Welcome Back to my website";
Integer visitCount = new Integer(0);
String visitCountKey = new String("visitCount");
String userIDKey = new String("userID");
String userID = new String("ABCD");

// Check if this is new comer on your web page.
if (session.isNew()) {
  title = "Welcome to my website";
  session.setAttribute(userIDKey, userID);
} else {
  visitCount = (Integer)session.getAttribute(visitCountKey);
  visitCount = visitCount + 1;
  userID = (String)session.getAttribute(userIDKey);
```

```
}
session.setAttribute(visitCountKey,  visitCount);

// Set response content type
response.setContentType("text/html");
PrintWriter out = response.getWriter();

String docType =
  "<!doctype html public \"-//w3c//dtd html 4.0 " +
  "transitional//en\">\n";

out.println(docType +
  "<html>\n" +
    "<head><title>" + title + "</title></head>\n" +

    "<body bgcolor = \"#f0f0f0\">\n" +
      "<h1 align = \"center\">" + title + "</h1>\n" +
      "<h2 align = \"center\">Session Infomation</h2>\n" +
      "<table border = \"1\" align = \"center\">\n" +

        "<tr bgcolor = \"#949494\">\n" +
          "  <th>Session info</th><th>value</th>
        </tr>\n" +
```

```
"<tr>\n" +
   "  <td>id</td>\n" +
   "  <td>" + session.getId() + "</td>
</tr>\n" +

"<tr>\n" +
   "  <td>Creation Time</td>\n" +
   "  <td>" + createTime + "  </td>
</tr>\n" +

"<tr>\n" +
   "  <td>Time of Last Access</td>\n" +
   "  <td>" + lastAccessTime + "  </td>
</tr>\n" +

"<tr>\n" +
   "  <td>User ID</td>\n" +
   "  <td>" + userID + "  </td>
</tr>\n" +

"<tr>\n" +
   "  <td>Number of visits</td>\n" +
```

```
                "  <td>" + visitCount + "</td>
              </tr>\n" +
            "</table>\n" +
          "</body>
        </html>"
    );
  }
}
```

Compile the above servlet **SessionTrack** and create appropriate entry in web.xml file. Now running *http://localhost:8080/SessionTrack* would display the following result when you would run for the first time −

Welcome to my website

Session Infomation

Session info	value
id	0AE3EC93FF44E3C525B4351B77ABB2D5
Creation Time	Tue Jun 08 17:26:40 GMT+04:00 2010
Time of Last Access	Tue Jun 08 17:26:40 GMT+04:00 2010
User ID	ABCD

Number of visits	0

Now try to run the same servlet for second time, it would display following result.

Welcome Back to my website

Session Infomation

info type	value
id	0AE3EC93FF44E3C525B4351B77ABB2D5
Creation Time	Tue Jun 08 17:26:40 GMT+04:00 2010
Time of Last Access	Tue Jun 08 17:26:40 GMT+04:00 2010
User ID	ABCD
Number of visits	1

Deleting Session Data

When you are done with a user's session data, you have several options −

- **Remove a particular attribute** − You can call *public void removeAttribute(String name)* method to delete the value associated with a particular key.
- **Delete the whole session** − You can call *public void invalidate()* method to discard an entire session.

- **Setting Session timeout** − You can call *public void setMaxInactiveInterval(int interval)* method to set the timeout for a session individually.
- **Log the user out** − The servers that support servlets 2.4, you can call **logout** to log the client out of the Web server and invalidate all sessions belonging to all the users.
- **web.xml Configuration** − If you are using Tomcat, apart from the above mentioned methods, you can configure session time out in web.xml file as follows.

```
<session-config>
  <session-timeout>15</session-timeout>
</session-config>
```

The timeout is expressed as minutes, and overrides the default timeout which is 30 minutes in Tomcat.

The getMaxInactiveInterval( ) method in a servlet returns the timeout period for that session in seconds. So if your session is configured in web.xml for 15 minutes, getMaxInactiveInterval( ) returns 900.